Librerías Multimedia

Unidad de Trabajo 3

Contenidos

1. Entornos de desarrollo multimedia
2. Gestión de datos multimedia
3. Conversión de formatos multimedia
4. Procesamiento de datos multimedia
5. Control de eventos, tipos de media y excepciones
6. Creación y control de animaciones
7. Reproducción de contenidos multimedia

Entornos de desarrollo multimedia

- **Multimedia**:
  - Audio
  - Vídeo
  - Imagen
  - Animación

- Entornos y frameworks multimedia más usados:
  - **Android SDK** (MediaPlayer, MediaRecorder, CameraX, Animation, ExoPlayer)
  - JavaFX (AudioClip, Media, MediaPlayer)
  - Unity Engine

Entornos de desarrollo multimedia

- **Ventajas de Android** como entorno multimedia:
  - Acceso a sensores y hardware.
  - Integración con eventos táctiles.
  - Soporte nativo de múltiples formatos.

- Proporciona **herramientas para la reproducción multimedia**, p.e.:
  - MediaPlayer → API nativa del SDK de Android
  - ExoPlayer → librería externa mantenida por Google

  Ambas permiten reproducir audio y vídeo, pero están pensadas para **escenarios distintos**.

Entornos de desarrollo multimedia

Característica	MediaPlayer	ExoPlayer
Tipo	API nativa	Librería externa
Facilidad de uso	Muy alta	Media
Reproducción local	Excelente	Excelente
Streaming	Básico	Avanzado
Control del buffer	Limitado	Completo
Gestión de errores	Básica	Avanzada
Formatos adaptativos	No	Sí (HLS, DASH)
Uso recomendado	Apps simples	Apps profesionales

MediaPlayer es perfecto para empezar/entender la reproducción multimedia en Android, mientras que ExoPlayer es la adecuada cuando la aplicación requiere streaming, robustez y control avanzado.

Gestión de datos multimedia

- Captura de audio:
  - MediaRecorder
- Captura de imagen/vídeo:
  - CameraX
  - Intents del sistema (ACTION_IMAGE_CAPTURE)
- Almacenamiento:
  - Almacenamiento interno de la app
  - Almacenamiento multimedia compartido (MediaStore)
- Permisos en Android (RECORD_AUDIO, CAMERA, READ_MEDIA_*)

MediaRecorder
✓ Graba audio desde el micrófono
✓ Codifica vídeo
✓ Guarda en MP4, 3GP, etc.

CameraX
✓ Captura fotos
✓ Graba vídeo
✓ Preview en tiempo real
✓ Gestión automática de rotación
✓ Compatibilidad entre dispositivos

Gestión de datos multimedia

- Captura de audio con MediaRecorder
  - **Audio Source** → Define de dónde viene el audio que quieres grabar (p.e.: Mic)
  - **Output Format** → contenedor del archivo de audio

Formato	Extensión típica	Características
MPEG_4	.mp4	Contenedor moderno, compatible con AAC, buena calidad, recomendado
THREE_GPP	.3gp	Antiguo, compatible con dispositivos antiguos, suele usar AMR
AAC_ADTS	.aac	Solo audio AAC, contenedor simple
AMR_NB / AMR_WB	.amr	Voice only, baja calidad, pequeño tamaño

Gestión de datos multimedia

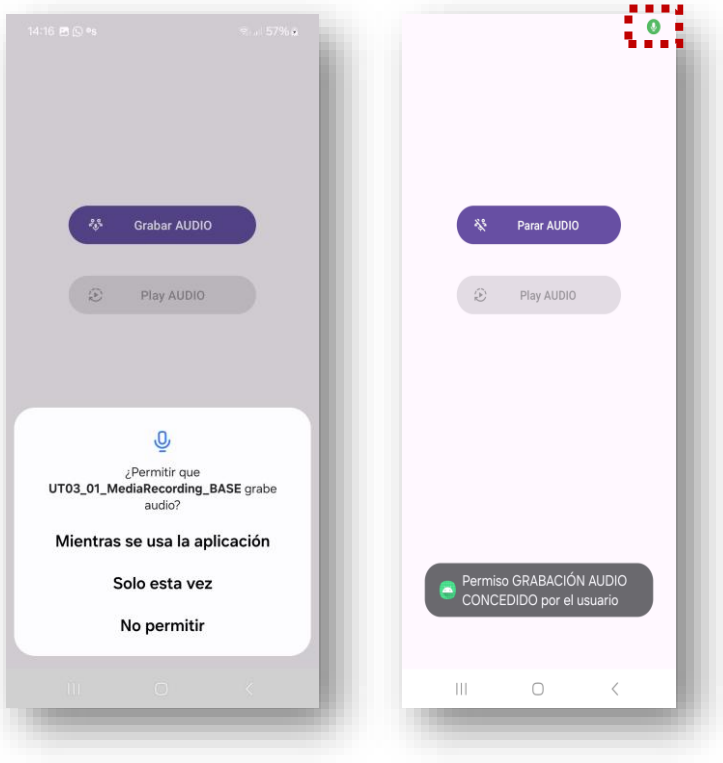
- Captura de audio con MediaRecorder
  - **Audio Encoder** → contenedor del archivo de audio

Encoder	Extensión típica	Características	Uso recomendado
AAC	.mp4	Alta calidad, compresión eficiente, estándar moderno	Grabación de voz y música, compatible con la mayoría de reproductores
AMR_NB	.3gp	Narrowband, voz solamente, baja calidad, pequeño tamaño	Aplicaciones antiguas o llamadas de voz
AMR_WB	.3gp	Wideband, voz, mayor calidad que AMR_NB	Apps de voz con mejor fidelidad
HE_AAC	.m4a	AAC con alta eficiencia a bitrates bajos	Streaming o apps que necesiten archivos pequeños
VORBIS	.ogg	Codec abierto, buena calidad, requiere contenedor OGG	Solo en Android 5+ y formatos OGG
DEFAULT	Depende del sistema	Codec por defecto del contenedor	Para casos simples de prueba

Gestión de datos multimedia

- Captura de audio con **MediaRecorder**

```
grabadorAudio= new MediaRecorder();

// Configuramos el origen, formato y codificación del audio
grabadorAudio.setAudioSource(MediaRecorder.AudioSource.MIC);
grabadorAudio.setOutputFormat(MediaRecorder.OutputFormat.MPEG_4);
grabadorAudio.setAudioEncoder(MediaRecorder.AudioEncoder.AAC);

// Preparamos el fichero para la grabación del audio
String ficheroAudio= getFilesDir() + "/" + audioFileName;
grabadorAudio.setOutputFile(ficheroAudio);
try {
   grabadorAudio.prepare();
} catch (IOException e) {
   throw new RuntimeException(e);
}

// Activamos la grabación por el micrófono del móvil
grabadorAudio.start();
```




```
// Paramos la grabación por el micrófono del móvil
if (grabadorAudio != null) {
   grabadorAudio.stop();
   grabadorAudio.release();
   grabadorAudio = null;
}
```

Gestión de datos multimedia

- Captura de audio con **MediaRecorder**
  - A tener en cuenta los permisos:

AndroidManifest.xml

```
<uses-permission android:name="android.permission. RECORD_AUDIO" />
```

SOLICITAR al usuario y gestionar respuesta en onRequestPermissionsResult

Gestión de datos multimedia

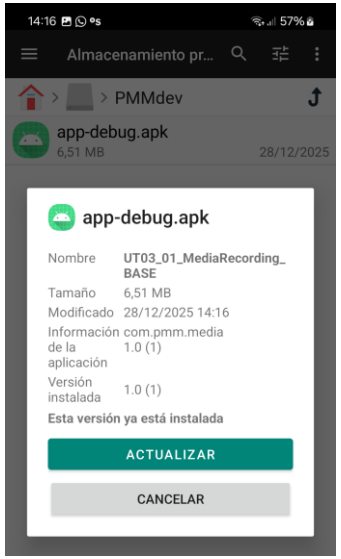
- Reproducción de audio con **MediaPlayer**:

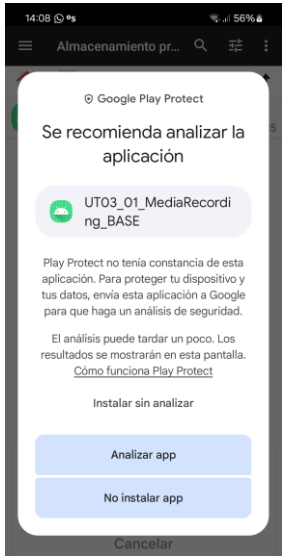
```
try {
    reproductorAudio = new MediaPlayer();

    // Preparamos el fichero para la reproducción del audio
    String ficheroAudio= getFilesDir() + "/" + audioFileName;
    reproductorAudio.setDataSource(ficheroAudio); // ruta completa del fichero
    reproductorAudio.prepare();
    reproductorAudio.start();

    // Configuramos un listener para que libere los recursos
    reproductorAudio.setOnCompletionListener(mp -> {
        mp.release();
        reproductorAudio = null;
        runOnUiThread(() -> btnPlayAudio.setEnabled(true));
    });

} catch (IOException e) {
    e.printStackTrace();
}
```

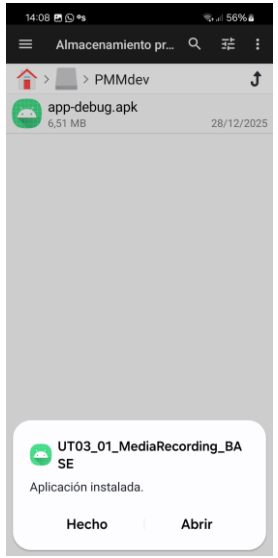
***NOTA**: Si ejecutamos la captura del audio en un emulador (AVD) de Android Studio podría capturar audio del micrófono del ordenador, pero puede fallar (o no tener micro).*

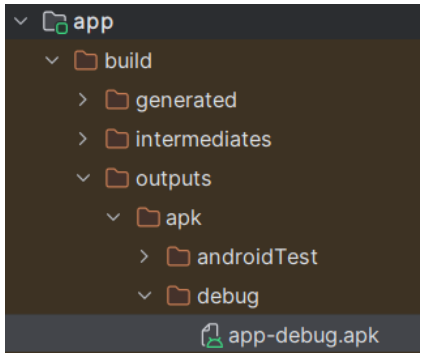
Gestión de datos multimedia

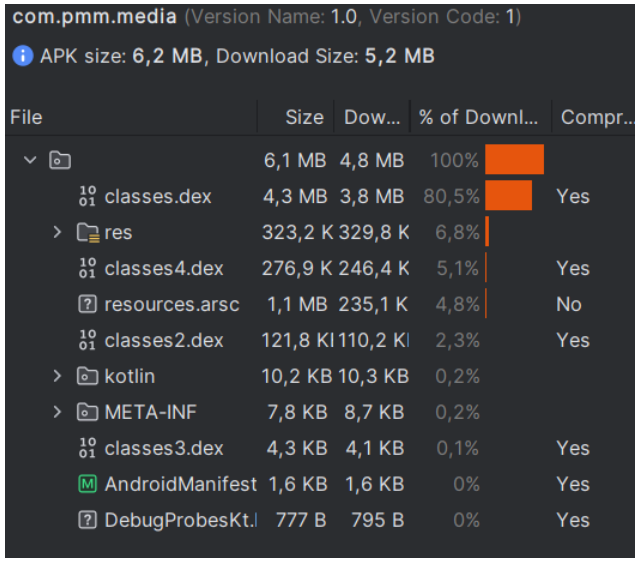
- Probar app en dispositivos móviles:
  - Conectar móvil con depuración USB habilitada. Android Studio lo tratará como dispositivo virtual.
  - Distribuir vía tienda Google Store creando una cuenta como desarrollador (25$).
  - Transferir la app al dispositivo móvil:
    - Generar un paquete APK en Android Studio: Build → Build Bundle(s) / APK(s) → Build APK(s)
    - Copiarlo al dispositivo móvil
    - Ejecutar el .apk en el móvil











Gestión de datos multimedia

- Captura de imágenes (formatos)

Formato	Extensión	Descripción	Uso recomendado
JPEG (JPG)	.jpg	Compresión con pérdidas	**Formato por defecto**
PNG	.png	Sin pérdidas	Capturas estáticas / gráficos
WEBP	.webp	Moderno, eficiente	Android moderno
HEIF / HEIC	.heic	Alta calidad, menos tamaño	Dispositivos recientes

  - Intentes y CameraX usan automáticamente jpeg siempre.

Gestión de datos multimedia

- Captura de imagen con Intent
  - Mediante Intents del sistema:
    - Se delega la captura en una **app de cámara externa** instalada en el dispositivo.
    - Nuestra app **no controla la cámara**, solo **recibe el resultado**.
  - Permisos:

AndroidManifest.xml

```
<uses-permission android:name="android.permission.CAMERA" />
```

Gestión de datos multimedia

- Captura de imagen con Intent
  - Debemos configurar un provider para intercambiar imágenes con la app externa

AndroidManifest.xml

```xml
<provider
   android:name="androidx.core.content.FileProvider"
   android:authorities="${applicationId}.fileprovider"
   android:exported="false"
   android:grantUriPermissions="true">
   <meta-data
      android:name="android.support.FILE_PROVIDER_PATHS"
      android:resource="@xml/file_paths"/>
</provider>
```

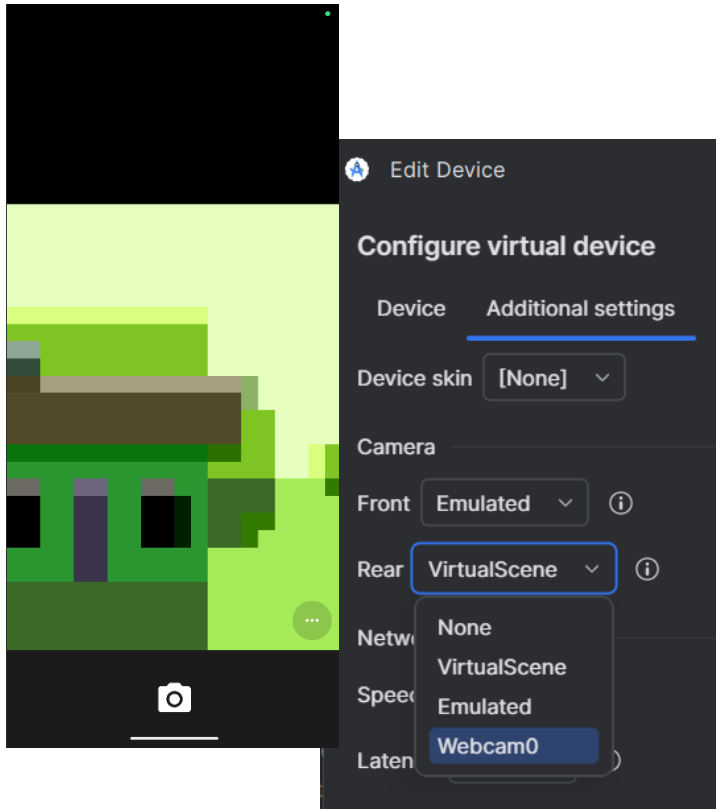
res/xml/file_paths.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<paths xmlns:android="http://schemas.android.com/apk/res/android">
   <external-files-path
      name="images"
      path="Pictures" />
</paths>
```

Gestión de datos multimedia

- Captura de imagen con Intent

```
// Establecemos la ruta donde nuestra app puede compartir ficheros, en particular el
// directorio para imágenes, y el nombre (con timestamp) de la imagen que será compartida.
String timeStamp = new SimpleDateFormat("yyyyMMdd_HHmmss", Locale.getDefault())
        .format(new Date());
String ficheroImagen= "IMG_" + timeStamp;
File storageDir = getExternalFilesDir(Environment.DIRECTORY_PICTURES);
File photoFile = new File(storageDir, ficheroImagen);

// Permite compartir ficheros dentro de /data/<nuestro_paquete>/files/Pictures/
// Conforme se define en el AndroidManifest <provider> y en el file_paths.xml
photoUri = FileProvider.getUriForFile(
        this,
        getPackageName() + ".fileprovider",
        photoFile
);
// Lanzamos el Intent
Intent intent = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);
intent.putExtra(MediaStore.EXTRA_OUTPUT, photoUri);
cameraLauncher.launch(intent);
```



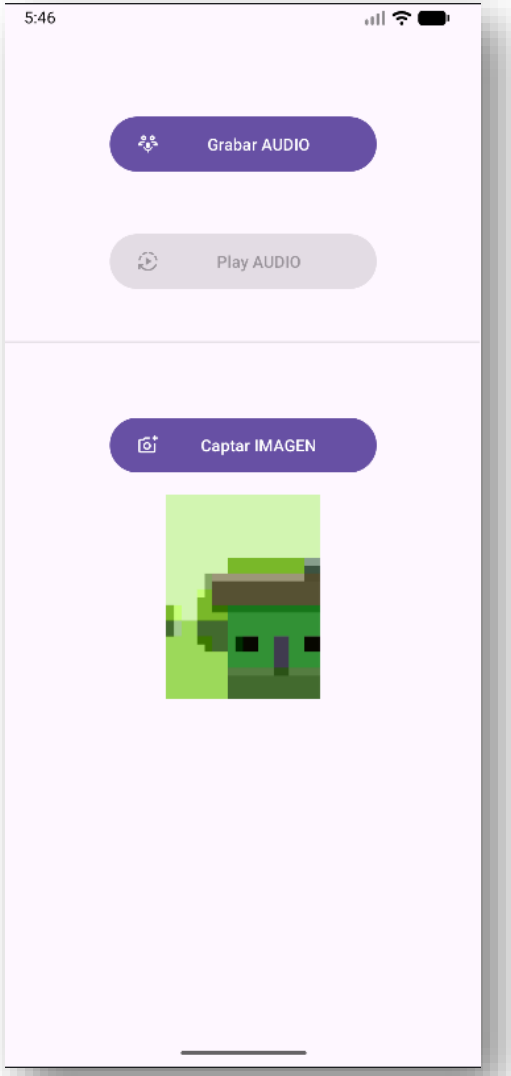
NOTA: *Podemos configurar si el AVD emula la cámara, utiliza una escena virtual o utiliza la cámara integrada en el ordenador.*

Gestión de datos multimedia

- Captura de imagen con Intent

```
// Vamos a monitorizar el resultado del Intent para mostrar la imagen obtenida en nuestra app
imgCaptura= findViewById(R.id.imgCaptura);
cameraLauncher= registerForActivityResult(
        new ActivityResultContracts.StartActivityForResult(),
        result -> {
            if (result.getResultCode() == Activity.RESULT_OK) {
                imgCaptura.setImageURI(photoUri);
            }
        });
```

  - La app no accede directamente a la cámara.
  - La app que realmente usa la cámara es la app del sistema.
  - En algunas circunstancias, Android maneja todos los permisos internamente.



Gestión de datos multimedia

- Conversión de datos multimedia de un formato a otro
  - Permite adaptar el formato de un fichero multimedia capturado o almacenado a otro que nos interese manejar por razones como:
    - Reducir tamaño
    - Compatibilidad
    - Subida a red
    - Canales alfa

```
File ficheroJpeg= new File(getExternalFilesDir(Environment.DIRECTORY_PICTURES), imageFileName);
File ficheroPng= new File(getFilesDir(), imageFileName.split("\\.")[0]+".png");

// Decodificar imagen JPEG a Bitmap
Bitmap bitmap = BitmapFactory.decodeFile(ficheroJpeg.getAbsolutePath());

// Crear flujo de salida para el PNG
fos = new FileOutputStream(ficheroPng);
// Comprimir y convertir a PNG
bitmap.compress(Bitmap.CompressFormat.PNG, 90, fos);
fos.flush();
// Cerrar flujo
fos.close();
```

Gestión de datos multimedia

- Captura de vídeo (formatos)

Contenedor	Extensión	Vídeo	Audio	Uso recomendado
MP4	.mp4	H.264 / H.265	AAC	Recomendado siempre
3GP	.3gp	H.263 / H.264	AMR	Apps antiguas / bajo peso
WEBM	.webm	VP8 / VP9	Vorbis / Opus	Apps modernas / open source

  - CameraX usa automáticamente mp4 siempre.
  - Con MediaRecorder se puede establecer la configuración con:

```
recorder.setOutputFormat(MediaRecorder.OutputFormat.MPEG_4);
recorder.setVideoEncoder(MediaRecorder.VideoEncoder.H264);
recorder.setAudioEncoder(MediaRecorder.AudioEncoder.AAC);
```

Gestión de datos multimedia

- Captura de vídeo con CameraX
  - CameraX es la librería oficial moderna de Android para usar la cámara
    - Simplifica la Camera API
    - Compatible con la mayoría de dispositivos
    - Integrada con ciclo de vida.
  - Dependencias:

app/build.gradle

```
// CameraX core
implementation("androidx.camera:camera-camera2:1.5.2")
// Integración con Lifecycle (para bindToLifecycle)
implementation("androidx.camera:camera-lifecycle:1.5.2")
// Vista de cámara (PreviewView)
implementation("androidx.camera:camera-view:1.5.2")
```

Gestión de datos multimedia

- Captura de vídeo con CameraX
  - Permisos:

    AndroidManifest.xml

```
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.RECORD_AUDIO" />
```

  - Características (no es obligatorio declararlas):

```
<uses-feature
  android:name="android.hardware.camera"
  android:required="false" />
```

required=“true"
✓ Solo se instala en dispositivos con cámara
✓ Ideal para apps centradas en foto/vídeo

required="false"
✓ La app se puede instalar, aunque no haya cámara
✓ Tú decides en tiempo de ejecución si usarla

Gestión de datos multimedia

- Captura de vídeo con CameraX
  - Atributos:

    MainActivity.java

    ```
    private PreviewView previewView;
    private VideoView videoView;
    private VideoCapture<Recorder> videoGrabador;
    private Recording grabacionVideo;
    private ProcessCameraProvider cameraProvider;
    private final static String videoFileName= "video_file.mp4";
    ```

  - Pasos:
    - Configurar Preview (para ver lo que se graba)
    - Configurar VideoCapture (objeto que define cómo grabar)
    - Enlazar al Lifecycle del proveedor de la cámara en el dispositivo
    - Iniciar la grabación mediante startRecording()
    - Detener la grabación mediante stopRecording()

Gestión de datos multimedia

- Captura de vídeo con CameraX (configuración de la cámara)

MainActivity.java

```
// Preview
PreviewView previewView = findViewById(R.id.previewViewVideo);
Preview preview = new Preview.Builder().build();
// VideoCapture
Recorder recorder = new Recorder.Builder().setQualitySelector(QualitySelector.from(Quality.HD)).build();
videoGrabador = VideoCapture.withOutput(recorder);
// CameraProvider
ListenableFuture<ProcessCameraProvider> cameraProviderFuture = ProcessCameraProvider.getInstance(this);
cameraProviderFuture.addListener(() -> {
    try {
        ProcessCameraProvider cameraProvider = cameraProviderFuture.get();
        CameraSelector cameraSelector = CameraSelector.DEFAULT_BACK_CAMERA;
        // Limpiar bindings previos
        cameraProvider.unbindAll();
        // Enlazar lifecycle. Hace a CameraX el dueño de la superficie del PreviewView.
        cameraProvider.bindToLifecycle(this,cameraSelector,preview,videoGrabador);
        preview.setSurfaceProvider(previewView.getSurfaceProvider());
    } catch (ExecutionException | InterruptedException e) {
        e.printStackTrace();
    }
}, ContextCompat.getMainExecutor(this));
```

Gestión de datos multimedia

- Captura de vídeo con CameraX (inicio/parada de la grabación)

iniciarGrabacionVideo

```
String ficheroVideo= getFilesDir() + "/" + videoFileName;
File videoFile = new File(ficheroVideo);

FileOutputOptions outputOptions =
      new FileOutputOptions.Builder(videoFile).build();

// Start
grabacionVideo = videoGrabador.getOutput()
      .prepareRecording(this, outputOptions)
      .withAudioEnabled() // requiere RECORD_AUDIO
      .start(ContextCompat.getMainExecutor(this),
            event -> {
               if (event instanceof VideoRecordEvent.Start) {
                  Log.d("VIDEO", "Grabación iniciada");
               } else if (event instanceof VideoRecordEvent.Finalize) {
                  Log.d("VIDEO", "Grabación finalizada");
               }
            });
```

pararGrabacionVideo

```
grabacionVideo.stop();
```

Gestión de datos multimedia

- Reproducción de vídeo con CameraX

reproducirVideo

```
runOnUiThread(() -> {
        // Detener cámara
        cameraProvider.unbindAll();
        previewView.setVisibility(View.GONE);

        // Mostrar VideoView
        videoView = findViewById(R.id.videoView);
        videoView.setVisibility(View.VISIBLE);

        // Reproducir video
        String ficheroVideo= getFilesDir() + "/" + videoFileName;
        videoView.setVideoURI(Uri.fromFile(new File(ficheroVideo)));
        videoView.start();
    });
```

pararReproduccionVideo

```
videoView.stopPlayback();
runOnUiThread(() -> {
   previewView.setVisibility(View.VISIBLE);
   videoView.setVisibility(View.GONE);
});
activarCamara();
```

Creación y control de animaciones

- Una animación es una **modificación progresiva de propiedades visuales** de un elemento gráfico a lo largo del tiempo.

- En Android se basan en:
  - Cambios de posición
  - Cambios de tamaño
  - Cambios de rotación
  - Cambios de opacidad

Creación y control de animaciones

- **Animaciones de vista (android.view.animation)**
  - Afectan solo a la representación visual
  - No modifican la posición/propiedad real del objeto
  - Definidas en ficheros XML ad-hoc dentro de res/anim/
- Ejemplos:
  - TranslateAnimation
  - ScaleAnimation
  - RotateAnimation
  - AlphaAnimation
- Uso típico:
  - Efectos simples
  - Retrocompatibilidad

activity_main.xml

```
android:visibility="invisible"
```

res/anim/fade_in.xml

```
<?xml version="1.0" encoding="utf-8"?>
<Alpha xmlns:android="http://schemas.android.com/apk/res/android"
   android:fromAlpha="0.0"
   android:toAlpha="1.0"
   android:duration="500" />
```

MainActivity.java

```
// Hacer visible antes de animar
btnGrabarVideo.setVisibility(View.VISIBLE);
// Cargar animación
Animation fadeIn = AnimationUtils.loadAnimation(this, R.anim.fade_in);
// Iniciar animación
btnGrabarVideo.startAnimation(fadeIn);
```

Creación y control de animaciones

- **Animaciones de propiedades (android.animation)**
  - Modifican propiedades reales del objeto
  - Mayor control y flexibilidad
  - Recomendadas actualmente
- Clases principales:
  - ObjectAnimator
  - ValueAnimator
  - AnimatorSet
- Uso típico:
  - Interfaces modernas
  - Animaciones encadenadas

activity_main.xml

```
android:enabled="false"
android:alpha="0"
```

MainActivity.java

```
btnGrabarVideo.animate()
    .alpha(1f)
    .setDuration(500)
    .setStartDelay(100)
    .start();
```

- view.animate() devuelve un ViewPropertyAnimator
- ViewPropertyAnimator es un wrapper de alto nivel
- Internamente Android utiliza ObjectAnimator
- alpha=0 (invisible) vs alpha=1 (totalmente visible)